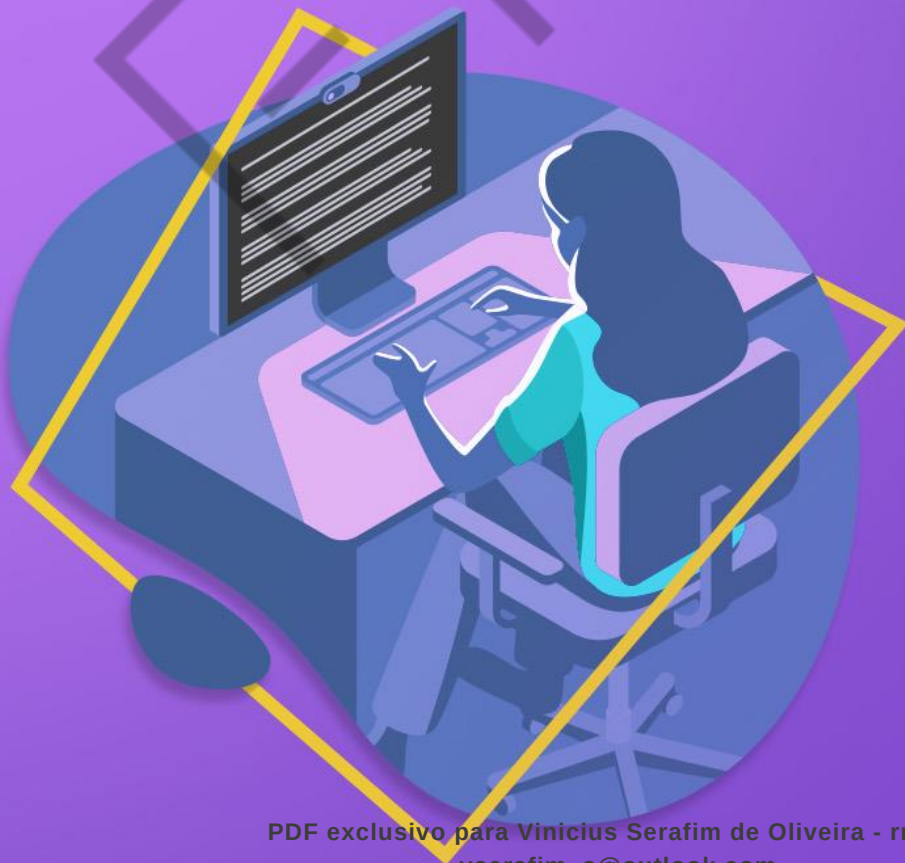


HACKER ATTACK

# CONHECENDO MAIS O CÓDIGO



3

## LISTA DE FIGURAS

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Figura 1 – Ataques SQL Injection e Cross-Site Scripting (XSS)..... | 5  |
| Figura 2 – Processo de Compilação .....                            | 11 |
| Figura 3 – Interface do AFL.....                                   | 12 |
| Figura 4 – Saídas do AFL .....                                     | 13 |

EMANIP

**LISTA DE CÓDIGOS-FONTE**

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Código-fonte 1 – Instalação dos pré-requisitos .....                  | 9  |
| Código-fonte 2 – Estrutura de diretórios da instalação do AFL .....   | 10 |
| Código-fonte 3 – Instalação do AFL .....                              | 10 |
| Código-fonte 4 – Download do código-fonte do binário de exemplo ..... | 10 |
| Código-fonte 5 – Compilação do binário de exemplo .....               | 11 |
| Código-fonte 6 – Sintaxe do AFL .....                                 | 11 |
| Código-fonte 7 – Sintaxe do AFL para o cenário proposto.....          | 12 |

EXEMPLO

## SUMÁRIO

|                                                  |    |
|--------------------------------------------------|----|
| FUZZING.....                                     | 5  |
| 1 FUZZING.....                                   | 5  |
| 2 DUMB FUZZERS .....                             | 6  |
| 3 TARGETED FUZZERS .....                         | 6  |
| 4 FUZZERS ACIONADOS POR FEEDBACK .....           | 6  |
| 5 MAIS PROTEÇÕES .....                           | 7  |
| 5.1 Conceitos básicos do American Fuzzy Lop..... | 7  |
| 6 O QUE É INSTRUMENTAÇÃO DE CÓDIGO? .....        | 8  |
| 7 LAB – CONSTRUINDO UM AMBIENTE DE FUZZING.....  | 9  |
| 8 MAIS PROTEÇÕES .....                           | 13 |
| REFERÊNCIAS.....                                 | 14 |

## FUZZING

### 1 FUZZING

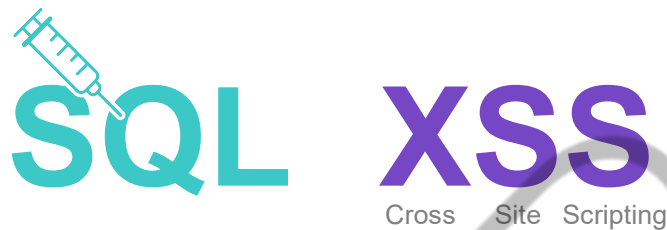


Figura 1 – Ataques SQL Injection e Cross-Site Scripting (XSS)  
Fonte: Google Imagens (2019)

Para ampliar a segurança de uma aplicação, ou da plataforma onde rodam as aplicações, muitos testes devem ser realizados e muitas observações e boas práticas devem ser seguidas. Neste capítulo, apresentaremos uma forma eficiente para você realizar um destes testes, estamos falando da técnica “fuzzing”. Seguimos ampliando os seus “skills”.

Fuzzing é um termo usado para descrever uma técnica de teste focada na identificação de anomalias em um aplicativo, através da avaliação de como o aplicativo responde quando apresentado com entradas diferentes. O conceito de Fuzzing não é novo e é muito eficaz quando empregado durante uma análise de vulnerabilidades e/ou em um Pentest em aplicações web, como SQL Injection ou Cross-Site Scripting (XSS).

Realizar o Fuzzing de um binário é uma técnica extremamente eficaz para identificar erros no código, que, de outra forma, passariam despercebidos, e é um primeiro passo dado pelos pesquisadores de segurança para descobrir vulnerabilidades Zero Day.

O conceito de Fuzzing é bem simples. Ao fazer o Fuzzing em um binário, você o executa repetidamente milhares de vezes, fornecendo sempre um arquivo de entrada ligeiramente diferente. De maneira não natural, você deseja que seus arquivos de entrada causem falha no destino, porque falhas tendem a significar corrupção de memória e, às vezes, a corrupção de memória pode levar a algo mais sinistro, como a execução remota de código.

Atualmente, existem três tipos principais de Fuzzing em uso: Dumb Fuzzers, Targeted Fuzzers e Feedback Driven Fuzzers.

## 2 DUMB FUZZERS

Sua ideia básica é alimentar o aplicativo de destino com entrada mutada aleatoriamente. Por exemplo, modifique aleatoriamente uma imagem jpeg e execute-a na biblioteca de conversão de imagens, esperando que falhe. Apesar de ser simples de usar e extremamente rápido, uma falha bem-sucedida depende muito da sorte da mutação aleatória.

Os Dumb Fuzzers não são adequados para testar aplicativos que verificam a integridade do conteúdo, pois o arquivo mutado aleatoriamente quase sempre quebra a soma de verificação. Outra questão é que os Dumb Fuzzers não podem ajudar no entendimento profundo do motivo do acidente, pois simplesmente jogam bits aleatórios e esperam o acidente. O exemplo mais comum é o **zzuf**.

## 3 TARGETED FUZZERS

Como o nome já diz, esse tipo de Fuzzing se baseia em partes muito específicas do aplicativo. Por exemplo, se queremos realizar o Fuzzing em arquivos rar, podemos usar Targeted Fuzzers, executando mutações aleatórias no conteúdo do arquivo, atualizando o valor da soma de verificação CRC.

Você pode aprofundar-se nesse tipo de Fuzzing e encontrar falhas muito interessantes, se souber onde procurar. Infelizmente, a configuração geralmente leva muito tempo, você precisa entender o formato e o aplicativo com os quais está lidando e escrever regras que dirão ao fuzzer o que fazer, quando e como. Alguns exemplos de Targeted Fuzzers são: **Peach Fuzz** e **Sulley**.

## 4 FUZZERS ACIONADOS POR FEEDBACK

Funcionam muito parecido com os Dumb Fuzzers, iniciando com mutações aleatórias nos dados de entrada. Mas, em vez de apenas registrar a saída da

operação, os fuzzers acionados por feedback se ajustam com base na saída, gerando famílias de tipos semelhantes de entrada mutada e tentando restringir a lista de possíveis entradas maliciosas necessárias para travar o programa.

Isso dá ao fuzzer a capacidade de se aprofundar no código e encontrar bugs muito ocultos. Os fuzzers acionados por feedback podem ser quase tão simples de usar quanto os Dumb Fuzzers. A desvantagem é o aquecimento, pois o fuzzer precisa encontrar algo para começar a aprender e direcionar as informações, e, até então, é apenas um Dumb Fuzzer, esperando o momento de “iniciar a brincadeira”. Exemplos: **American Fuzzing Lop** e **honggfuzz**.

O processo do Fuzzing é altamente automatizado e depende da ferramenta. A automação fornecida pelas ferramentas faz todo o processo de criação de milhares de casos de teste de entrada exclusivos e de execução e finalização repetitiva da prática binária de destino. Neste material, usaremos o American Fuzzy Lop (AFL) para nos ajudar em nossa jornada.

## 5 MAIS PROTEÇÕES

### 5.1 Conceitos básicos do American Fuzzy Lop

O American Fuzzy Lop (AFL) é um fuzzer de código aberto que tem sido fundamental para descobrir inúmeras vulnerabilidades em muitos dos pacotes de software populares da atualidade, como nginx, OpenSSH, OpenSSL e PHP, entre outros. Ele funciona em arquiteturas x86 Linux, OpenBSD, FreeBSD e NetBSD, de 32 e 64 bits. Ele suporta programas escritos em C, C ++, Objective C, compilados com gcc ou clang.

Seu processo de funcionamento é muito simples:

- Compile um binário usando os wrappers de compilador do AFL.
- Confunda o binário usando afl-fuzz (é aqui que a mágica acontece).
- Analise quaisquer falhas exclusivas relatadas pelo afl-fuzz. Falhas exclusivas ocorrem quando qualquer um dos arquivos de entrada modificados pelo afl-fuzz resulta em uma falha no binário de destino.

O fluxo de trabalho do afl-fuzz é o seguinte:

- O afl-fuzz pega um arquivo como entrada do PATH especificado (usando o parâmetro -i) e executa o binário de destino. Depois, monitora a atividade binária para a operação normal ou falha. Se nenhuma falha for detectada, o afl-fuzz encerra o binário e continua para o passo 2.
- O afl-fuzz faz uma pequena modificação no arquivo inicial e executa o binário de destino mais uma vez, usando esse novo arquivo como entrada, monitora a atividade e repete o ciclo.
- O afl-fuzz faz outra modificação menor no arquivo inicial e executa o binário de destino novamente exatamente da mesma maneira, usando o arquivo modificado como entrada, mais uma vez o afl-fuzz monitora a atividade. Neste exemplo em particular, o arquivo inicial modificado causa uma falha no binário! Se isso acontecer, o afl-fuzz colocará uma cópia do arquivo inicial (que causou a falha) no diretório/crashes dentro do PATH especificado (com o parâmetro -o). O afl-fuzz continuará modificando um teste e repetindo a partir da etapa 1.

## 6 O QUE É INSTRUMENTAÇÃO DE CÓDIGO?

A primeira etapa do processo descrito acima é uma etapa crítica, pois quando um binário é compilado com os wrappers do compilador do AFL para C ou C ++, o código-fonte é "instrumentado". A instrumentação se refere à capacidade de monitorar ou medir o nível de desempenho de um produto, diagnosticar erros e gravar informações de rastreamento.

De forma simplista, a "instrumentação", em nosso contexto, ocorrerá durante a compilação do código-fonte binário de destino. É o processo de inserir pequenas quantidades de código adicional no binário, de uma maneira que não perturbe as referências de memória usadas nas instruções do programa binário.

O código adicionado durante a instrumentação às vezes é chamado de "código de instrumentação" e é através desse código de instrumentação que alcançamos a capacidade de monitorar e medir o que está ocorrendo na memória,



enquanto o binário está em execução e, mais importante, quando ocorre uma falha.

## 7 LAB – CONSTRUINDO UM AMBIENTE DE FUZZING

Agora, vamos começar a criar o ambiente de Fuzzing, que será composto pelos seguintes componentes:

- Uma instalação limpa do Ubuntu 19.0.4.
- Pré-requisitos (gcc, clang, gdb).
- American Fuzzy Lop (AFL).

Vamos começar com a instalação dos pré-requisitos do nosso AFL:

```
$ sudo apt-get update  
$ sudo apt-get install build-essential  
$ sudo apt-get install clang  
$ sudo apt-get install gcc  
$ sudo apt-get install gdb
```

Código-fonte 1 – Instalação dos pré-requisitos  
Fonte: Elaborado pelo autor (2019)

Em seguida, vamos criar uma estrutura básica de diretórios para agilizar nossos esforços semelhantes ao abaixo. A estrutura mostrada é opcional e não é necessária; portanto, fique à vontade para usar uma estrutura de diretórios de sua escolha. Minha preferência pessoal é geralmente seguir uma estrutura semelhante a seguinte:

```
~/
|_AFL    <--- diretório principal do AFL
|_targets <--- diretório das aplicações-alvo
|_in     <--- diretório para os arquivos exemplo de entrada
|_out    <--- diretório de saída do AFL
```

Código-fonte 2 – Estrutura de diretórios da instalação do AFL  
Fonte: Elaborado pelo autor (2019)

O AFL pode ser obtido diretamente no repositório Ubuntu conforme abaixo:

```
$ sudo apt-get install afl
```

Código-fonte 3 – Instalação do AFL  
Fonte: Elaborado pelo autor (2019)

Como mencionado anteriormente, para executar o Fuzzing de um binário, precisamos primeiro compilar o código-fonte do aplicativo, usando os wrappers do compilador do AFL para instrumentar o binário. Vamos continuar o processo de Fuzzing, obtendo o código-fonte do nosso aplicativo de destino. O programa que está sendo analisado para fins de demonstração é um simples programa de conversão de gif para png (gif2png). A versão mais recente do código-fonte está disponível nos repositórios do Ubuntu.

```
$ cd ~/targets
$ apt-get source gif2png
$ cd gif2png
```

Código-fonte 4 – Download do código-fonte do binário de exemplo  
Fonte: Elaborado pelo autor (2019)

Agora que temos o código-fonte, precisamos compilá-lo usando o compilador afl-clang-fast. Isso garantirá que o binário seja suficientemente instrumentado. Como o aplicativo de destino possui um script de configuração automática (configure.sh), compilamos fornecendo configurações de compilação e usando o padrão "configure" e "make":

```
$ CC="afl-clang-fast" CFLAGS="-fsanitize=address -ggdb" CXXFLAGS="-fsanitize=address -ggdb" ./configure
```

```
$ make
```

Código-fonte 5 – Compilação do binário de exemplo  
Fonte: Elaborado pelo autor (2019)

Se tudo correr bem, a saída da tela de [make] deve mostrar uma instrumentação bem-sucedida ocorrendo durante o processo de compilação – semelhante à imagem mostrada abaixo:

```
make[1]: Entering directory
afl-clang-fast -DHAVE_CONFIG_H -I. -fsanitize=address -ggdb -MT 437_l1.o -MD -MP -MF .deps/437_l1.Tpo -c -o 437_l1.o 437_l1.c
afl-clang-fast 2.36b by <lszekeres@google.com>
afl-llvm-pass 2.36b by <lszekeres@google.com>
[+] Instrumented 1 locations (non-hardened mode, ratio 100%).
mv -f .deps/437_l1.Tpo .deps/437_l1.o
afl-clang-fast -DHAVE_CONFIG_H -I. -fsanitize=address -ggdb -MT .deps/ -MD -MP -MF .deps/
afl-clang-fast 2.36b by <lszekeres@google.com>
afl-llvm-pass 2.36b by <lszekeres@google.com>
[+] Instrumented 307 locations (non-hardened mode, ratio 100%).
mv -f .deps/gif2png.Tpo .deps/
afl-clang-fast -DHAVE_CONFIG_H -I. -fsanitize=address -ggdb -MT .deps/ -MD -MP -MF .deps/
afl-clang-fast 2.36b by <lszekeres@google.com>
afl-llvm-pass 2.36b by <lszekeres@google.com>
[+] Instrumented 263 locations (non-hardened mode, ratio 100%).
mv -f .deps/gifread.Tpo .deps/
afl-clang-fast -DHAVE_CONFIG_H -I. -fsanitize=address -ggdb -MT memory.o -MD -MP -MF .deps/
afl-clang-fast 2.36b by <lszekeres@google.com>
afl-llvm-pass 2.36b by <lszekeres@google.com>
[+] Instrumented 37 locations (non-hardened mode, ratio 100%).
mv -f .deps/memory.Tpo .deps/memory.o
afl-clang-fast -DHAVE_CONFIG_H -I. -fsanitize=address -ggdb -MT version.o -MD -MP -MF .deps/
afl-clang-fast 2.36b by <lszekeres@google.com>
afl-llvm-pass 2.36b by <lszekeres@google.com>
[+] Instrumented 1 locations (non-hardened mode, ratio 100%).
mv -f .deps/version.Tpo .deps/version.o
afl-clang-fast -fsanitize=address -ggdb -o
afl-clang-fast 2.36b by <lszekeres@google.com>
memory.o version.o -lpng -lm -lz
```

Figura 2 – Processo de Compilação  
Fonte: Elaborado pelo autor (2019)

A etapa final da preparação é copiar um arquivo de entrada adequado para o diretório ~/targets/in. Quando inicializado, o afl-fuzz fornecerá inicialmente esse arquivo de entrada para o binário de destino. Selecione um arquivo de entrada da extensão apropriada e de tamanho pequeno e coloque-o no diretório ~/targets/in.

Finalmente, podemos começar o processo de Fuzzing. Como uma rápida recapitulação, instalamos e otimizamos o AFL com êxito, obtivemos e compilamos o binário de destino usando afl-clang-fast e colocamos casos de teste de entrada (arquivos .png e .gif) no diretório /in.

O passo final é iniciar o fuzzer na prática. O comando afl-fuzz é usado para executar o AFL, e a sintaxe completa para iniciar o afl-fuzz é a seguinte:

```
$ afl-fuzz -i [PATH TO TESTCASE_INPUT_DIR] -o [PATH TO OUTPUT_DIR] --
[PATH TO TARGET BINARY] [BINARY_PARAMS] @@
```

Código-fonte 6 – Sintaxe do AFL  
Fonte: Elaborado pelo autor (2019)

Em nosso cenário, seria:

```
$ cd ~/targets
```

```
$ afl-fuzz -i in/ -o out/ -- target-app_directory/target-app_binary @@
```

Código-fonte 7 – Sintaxe do AFL para o cenário proposto  
Fonte: Elaborado pelo autor (2019)

A partir desse instante, já iniciamos nosso Fuzzing e nosso objetivo nesse momento deverá ser o monitoramento do progresso do afl-fuzz, para garantir que o afl-fuzz encontre com êxito novos caminhos de execução de código no binário de destino e execute com eficiência. Durante a execução, o afl-fuzz exibe uma interface básica, que, por sua vez, mostra várias métricas para o usuário.

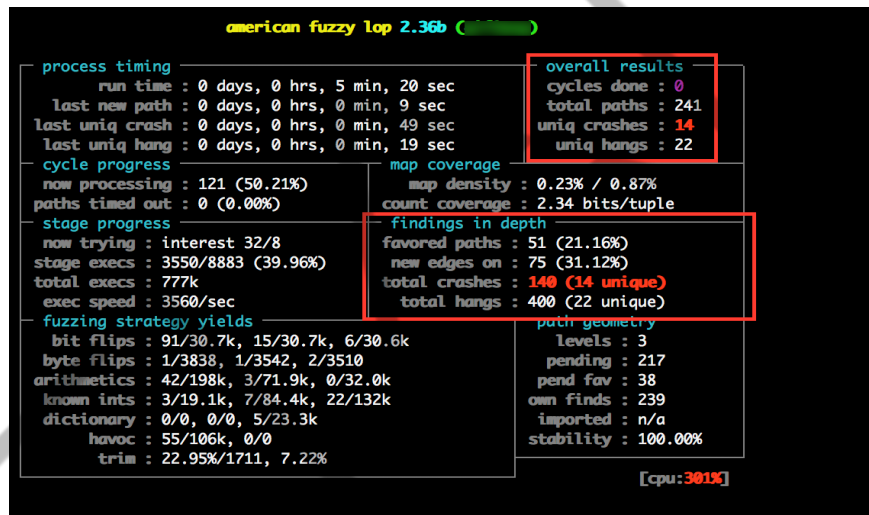


Figura 3 – Interface do AFL  
Fonte: Elaborado pelo autor (2019)

Depois que o AFL terminou sua “mágica”, é hora de começar a explorar cada falha encontrada. Durante a exploração de falhas, nosso objetivo é determinar por que e como o binário travou e qual é a propriedade exclusiva do arquivo de entrada de teste que causou o travamento.

O AFL armazena cada caso de teste que ele gera e alimenta no binário de destino no local especificado com a opção -o ao iniciar o afl-fuzz. Isso geralmente é chamado de local de saída. Casos de teste que resultaram em uma falha única são armazenados em um subdiretório em nosso ambiente de teste ~/targets/out/crashes.

```
~/targets/out$ ls
crashes  fuzz_bitmap  fuzzer_stats  hangs  plot_data  queue
~/targets/out$ cd crashes/
~/targets/out/crashes$ ls
id:000000,sig:11,src:000000,op:int32,pos:52,val:be:+32
id:000001,sig:11,src:000000,op:int32,pos:52,val:be:+64
id:000002,sig:11,src:000000,op:int32,pos:52,val:be:+100
id:000003,sig:11,src:000000,op:havoc,rep:8
id:000004,sig:11,src:000000,op:havoc,rep:4
id:000005,sig:11,src:000004,op:int32,pos:52,val:be:+32
id:000006,sig:11,src:000020,op:int32,pos:52,val:be:+32
id:000007,sig:11,src:000020,op:havoc,rep:16
id:000008,sig:11,src:000026,op:flip1,pos:55
id:000009,sig:11,src:000026,op:havoc,rep:32
id:000010,sig:11,src:000078,op:int32,pos:52,val:be:+32
id:000011,sig:11,src:000094,op:int32,pos:52,val:be:+32
id:000012,sig:11,src:000096,op:int32,pos:52,val:be:+32
id:000013,sig:11,src:000099,op:int32,pos:52,val:be:+32
id:000014,sig:11,src:000173,op:flip1,pos:81
id:000015,sig:11,src:000173,op:havoc,rep:32
id:000016,sig:11,src:000221,op:int32,pos:52,val:be:+32
id:000017,sig:11,src:000238,op:int32,pos:52,val:be:+32
id:000018,sig:11,src:000238,op:ext_A0,pos:53
id:000019,sig:11,src:000273,op:int32,pos:52,val:be:+32
README.txt
~/targets/out/crashes$
```

Figura 4 – Saídas do AFL  
Fonte: Elaborado pelo autor (2019)

Como pode ser visto na imagem acima, o AFL encontrou 28 pontos de falhas (depois de quase 12 horas rodando...), as quais deveriam ser corrigidas antes de colocar sua aplicação em produção.

## 8 MAIS PROTEÇÕES

Durante este ano você aprenderá sobre uma técnica de Pentest, chamada Buffer Overflow (BoF), que tem tudo a ver com Fuzzing. É através das análises de Fuzzing que conseguimos encontrar possíveis pontos de falhas em aplicações, as quais seriam suscetíveis a ataques do tipo BoF.

Do ponto de vista da Defesa Cibernética, uma análise de Fuzzing pode auxiliar na entrega de aplicações mais seguras e robustas. E é isso que esperamos que você faça junto ao cliente “FIAP COIN”.

Neste momento, se faz importante entender um pouco mais sobre Linguagem C e suas estruturas de dados. Você já viu estes assuntos no último ano, mas lembre-se que relembrar é viver, e este, sem dúvida, é um bom momento para fortalecer ainda mais estes conhecimentos, que serão muito úteis no decorrer deste ano.

## REFERÊNCIAS

AMERICAN FUZZY LOP. Disponível em: <<http://lcamtuf.coredump.cx/afl/>>. Acesso em: 11 dez. 2020.

EMULOP